

Exercise 1

Results for n = 10:

```
f=@integrand;
a=0; % lower limit
b=1; % upper limit
n=10;
trapezoidal = segment_trapezoidal(f,a,b,n);
built_in_integral = integral(f,a,b);
simpson = simpson_one_third(f,a,b,n);
true_value = 0.602297;
percent_error_trapezoidal = 100*abs(true_value-trapezoidal)/true_value;
percent_error_integral = 100*abs(true_value-built_in_integral)/true_value;
percent_error_simpson = 100*abs(true_value-simpson)/true_value;
fprintf(['trapezoidal = %4.5f \n' ...
        'built_in_integral = %4.5f \n' ...
        'simpson = %4.5f \n' ...
        'percent_error_trapezoidal = %4.5f \n' ...
        'percent_error_integral = %4.5f \n' ...
        'percent_error_simpson = %4.5f \n'],trapezoidal, built_in_integral,simpson,percent_error_trapezoidal,
percent_error_integral,percent_error_simpson)
```

```
trapezoidal = 0.55980
built_in_integral = 0.60230
simpson = 0.57738
percent_error_trapezoidal = 7.05652
percent_error_integral = 0.00018
percent_error_simpson = 4.13712
```

Results for n = 100:

```
f=@integrand;
a=0; % lower limit
b=1; % upper limit
n=100;
trapezoidal = segment_trapezoidal(f,a,b,n);
built_in_integral = integral(f,a,b);
simpson = simpson_one_third(f,a,b,n);
true_value = 0.602297;
percent_error_trapezoidal = 100*abs(true_value-trapezoidal)/true_value;
percent_error_integral = 100*abs(true_value-built_in_integral)/true_value;
percent_error_simpson = 100*abs(true_value-simpson)/true_value;
fprintf(['trapezoidal = %4.5f \n' ...
        'built_in_integral = %4.5f \n' ...
        'simpson = %4.5f \n' ...
        'percent_error_trapezoidal = %4.5f \n' ...
        'percent_error_integral = %4.5f \n' ...
        'percent_error_simpson = %4.5f \n'],trapezoidal, built_in_integral,simpson,percent_error_trapezoidal,
percent_error_integral,percent_error_simpson)
```

```
'percent_error_simpson = %4.5f \n'],trapezoidal, built_in_integral,simpson,percent_error_tr
percent_error_simpson)
```

```
trapezoidal = 0.59911
built_in_integral = 0.60230
simpson = 0.60034
percent_error_trapezoidal = 0.52914
percent_error_integral = 0.00018
percent_error_simpson = 0.32446
```

Results for n = 1000:

```
f=@integrand;
a=0; % lower limit
b=1; % upper limit
n=1000;
trapezoidal = segment_trapezoidal(f,a,b,n);
built_in_integral = integral(f,a,b);
simpson = simpson_one_third(f,a,b,n);
true_value = 0.602297;
percent_error_trapezoidal = 100*abs(true_value-trapezoidal)/true_value;
percent_error_integral = 100*abs(true_value-built_in_integral)/true_value;
percent_error_simpson = 100*abs(true_value-simpson)/true_value;
fprintf(['trapezoidal = %4.5f \n' ...
        'built_in_integral = %4.5f \n' ...
        'simpson = %4.5f \n' ...
        'percent_error_trapezoidal = %4.5f \n' ...
        'percent_error_integral = %4.5f \n' ...
        'percent_error_simpson = %4.5f \n'],trapezoidal, built_in_integral,simpson,percent_error_tr
percent_error_simpson)
```

```
trapezoidal = 0.60205
built_in_integral = 0.60230
simpson = 0.60214
percent_error_trapezoidal = 0.04153
percent_error_integral = 0.00018
percent_error_simpson = 0.02560
```

```
function [I] = segment_trapezoidal(f,a,b,n)

I=0; % initial value of integral
h=(b-a)/n;
for i=0:n
    x= a + i*h; % goin from a to b
    if (i==0 || i==n)
        I = I + f(x);
    else % if is not 0 or n
        I = I + 2*f(x);
    end
end
I = I*h/2;
end

function [y] = integrand(x) % integrand function
```

```

y=x.^(0.1).*(1.2-x).*(1-exp(20.*(x-1)));
end

function [I] = simpson_one_third(f,a,b,n)
    if (mod(n,2)==0)
        I=0;
        h = (b-a)/n;
        for i =0:n
            x = a + i*h;
            if (i==0||i==n)
                I = I + f(x);
            elseif (mod(i,2)==1)
                I = I + 4*f(x);
            else
                I = I + 2*f(x);
            end
        end

        I = I*h/3;
    else
        disp("n has to be even")
    end
end
end

```

Exercise 2:

```
beam_length = 12; % meters
g=@shear_force;
n = 24; % step size is 0.5m, implies n = 24
syms f(x);
f(x) = 5 + 0.25*x^2;
M_analytical = double(int(f,x,0,beam_length));
M_trapezoidal = trapezoidal_rule(g,0,beam_length,n);
M_simpson = simpson(g,0,beam_length,n);
fprintf("Bending moment M analytical= %4.3f \nBending moment M trapezoidal= %4.3f \n" + ...
        "Bending moment M simpson %4.3f \n", ...
        M_analytical,M_trapezoidal,M_simpson)
```

```
Bending moment M analytical= 204.000
Bending moment M trapezoidal= 204.125
Bending moment M simpson 202.313
```

```
function [V] = shear_force(x)
V = 5+ 0.25*x.^2;
end

function [I] = trapezoidal_rule(f,a,b,n)
I=0; % initial value of integral
h = (b-a)/n; % step size
for i=0:n
    x = a + i*h; % going from a to b with 0.5 step size
    if (i==0 || i==n)
        I = I + f(x);
    else
        I = I + 2*f(x);
    end
end
I = I*h/2;
end

function [I] = simpson(f,a,b,n)
if (mod(n,2)==0)
    h=(b-a)/n;
    I=0;
    for i=0:n
        x = a + i*h;
        if(i==0 || i==n)
            I = I + f(x);
        elseif (mod(i,2)==1 && i~=1)
            I = I + 4*f(x);
        else
            I = I + 2*f(x);
        end
    end
    I = I*h/3;
else
```

```
    disp("n must be even")  
end  
end
```

Exercise 3

```
a = 0; % lower limit
H = 30; % upper limit
n = 100; % number of segments
f1 = @(z)force_on_sailboat(z,H);
f2 = @(z)numerator(z,H);
F_integral = integral(f1,a,H);
d_integral = integral(f2,a,H)/integral(f1,a,H);
F_trapezoidal = trap(f1,a,H,n);
d_trapezoidal = trap(f2,a,H,n)/trap(f1,a,H,n);
F_simp = simp(f1,a,H,n);
d_simp = simp(f2,a,H,n)/simp(f1,a,H,n);
fprintf( ...
"Total Force F using integral = %4.4f \n" + ...
"Total Force F using trapezoidal = %4.4f \n" + ...
"Total Force F using simpson = %4.4f \n",F_integral,F_trapezoidal,F_simp)
```

```
Total Force F using integral = 970.4526
Total Force F using trapezoidal = 970.2316
Total Force F using simpson = 970.4523
```

```
fprintf("Line of Action d using integral = %4.4f \n" + ...
"Line of Action d using trapezoidal = %4.4f \n" + ...
"Line of Action d using simpson = %4.4f \n",d_integral,d_trapezoidal, d_simp)
```

```
Line of Action d using integral = 12.1164
Line of Action d using trapezoidal = 12.1191
Line of Action d using simpson = 12.1164
```

Comparing results of simpson and trapezoidal with built-in integral function for F:

```
perect_error_trapezoidal = 100*abs(F_trapezoidal-F_integral)/F_integral;
percent_error_simp = 100*abs(F_simp-F_integral)/F_integral;
fprintf("Percent error for trapezoidal = %f \n" + ...
"Percent error for simpson = %f \n" + ...
"We can see that the simpson 1/3 approximation, when compared with the trapezoidal function
"is closer to the built-in integral function result when calculating F.",perect_error_trapezoidal,percent_error_simp)
```

```
Percent error for trapezoidal = 0.022779
Percent error for simpson = 0.000030
We can see that the simpson 1/3 approximation, when compared with the trapezoidal function,
is closer to the built-in integral function result when calculating F.
```

Calculatin percent error for Line of Action d:

```
p_error_simp = 100*abs(d_integral-d_simp)/d_integral;
p_error_trap = 100*abs(d_integral-d_trapezoidal)/d_integral;
fprintf("Perecent error for Line of Action d trapezoidal = %f \n" + ...
"Percent of error for Line of Action d simpson = %f \n" + ...
"We can see that the approximation to the Line of action d given\n" + ...
"by the simpson function is closer to the integral function result\n" + ...
"with a percent error of %f",p_error_trap,p_error_simp,p_error_simp)
```

```
Perecent error for Line of Action d trapezoidal = 0.021729
```

Percent of error for Line of Action d simpson = 0.000045

We can see that the approximation to the Line of action d given by the simpson function is closer to the integral function result with a percent error of 0.000045

```
function [y] = force_on_sailboat(z,H)
y = 200*(z./(7+z)).*exp(-2.7*z./H);
end

function [y] = numerator(z,H)
y = z.*(200*(z./(7+z)).*exp(-2.7*z./H));
end

function [I] = trap(f,a,b,n)
I=0; % integral initial value
h = (b-a)/n; % step size
for i=1:n
    x = a + i*h;% going from a to b w. step size h
    if (i==0 || i==n)
        I = I + f(x); % evaluate end points
    else
        I = I + 2*f(x);
    end
end
I = I*h/2;
end

function [I] = simp(f,a,b,n)
if (mod(n,2)==0)
    I = 0;
    h = (b-a)/n;
    for i=1:n
        x = a + i*h;
        if (i==0 || i==n)
            I = I + f(x); % endpoints
        elseif (mod(i,2)==0)
            I = I + 2*f(x);
        else
            I = I + 4*f(x);
        end
    end
    I = I*h/3;
else
    disp("number of segments n must be even")
end
end
```

Exercise 4:

```
a = 0; % lower limit
syms f(r,R);
f(r,R) = r*2*(1-r/R)^(1/6);
R = 2; % cm and upper limit
f = subs(f);
Q_symbolic = 2*pi*int(f,r,a,R);
f1 = @(r)integrand(r,R);
Q_integral = integral(f1,a,R);
Q_simpson = simpson_onethird(f1,a,R,100);
fprintf("Results: \n" + ...
    "Flow Rate Q symbolic = %f \n" + ...
    "Flow Rate Q integral = %f \n" + ...
    "Flow Rate Q simpson 1/3 = %f",Q_symbolic, Q_integral,Q_simpson)
```

```
Results:
Flow Rate Q symbolic = 19.885246
Flow Rate Q integral = 19.885246
Flow Rate Q simpson 1/3 = 19.834620
```

Comparison:

```
p_error=100*abs(Q_simpson-Q_integral)/Q_integral;
fprintf("The result obtained from the symbolic integral, and the built in integral function is
    "However, simpsons 1/3 for 100 segments (n = 100) has a percent error of %f when compared
    "with the symbolic (analytical) and integral (built in) functions results.", p_error)
```

The result obtained from the symbolic integral, and the built in integral function is the same. However, simpsons 1/3 for 100 segments (n = 100) has a percent error of 0.254589 when compared with the symbolic (analytical) and integral (built in) functions results.

```
function [rV] = integrand(r,R)
rV = 2*pi*2*r.*(1-r/R).^(1/6);
end

function [I] = simpson_onethird(f,a,b,n)
if (mod(n,2)==0) % total number of segments is even
    I=0;
    h = (b-a)/n;
    for i=1:n
        x = a + i*h;
        if (i==0 || i==n)
            I = I + f(x); % endpoints
        elseif (mod(i,2)==0)
            I = I + 2*f(x);
        else
            I = I + 4*f(x);
        end
    end
    I = h*I/3;
else
    disp("n must be even")
end
```

end

Exercise 5:

```
a = 0; % lower limit
b=10; % upper limit
y = [0 1 3 5 7 8 9 10]; % distance from the bank
H = [0 1 1.5 3 3.5 3.2 2 0]; % depth
U=[0 0.1 0.12 0.2 0.25 0.3 0.15 0]; % velocity
Ac_numerical = tabular_trapezoidal(y,H);
H_bar = Ac_numerical/b; % average depth
Q_FlowRate = tabular_trapezoidal(y,H.*U);
U_bar = Q_FlowRate/Ac_numerical; % average velocity
fprintf("Cross sectional area Ac = %f \n" + ...
    "Average depth H_bar = %f \n" + ...
    "Flow rate Q = %f \n" + ...
    "Average velocity U_bar = %f \n",Ac_numerical,H_bar,Q_FlowRate,U_bar)
```

```
Cross sectional area Ac = 20.950000
Average depth H_bar = 2.095000
Flow rate Q = 4.282500
Average velocity U_bar = 0.204415
```

```
function [I] = tabular_trapezoidal(x,y) % x and y vectors
if (length(x)==length(y))
    I = 0;
    n = length(x);
    for i=1:n-1
        h = x(i+1)-x(i); % width of segment i
        I = I + h*(y(i+1)+y(i))/2;
    end
else
    disp("Vectors x and y must be same length.")
end
end

function [y]=linear_interpolation(X,Y,xq)
y = interp1(X,Y,xq);
end
```

Exercise 6:

```
syms f(x,y);
f(x,y) = x^2-3*y^2+x*y^3;
symbolic = double(int(int(f,y,0,4),-2,2));
Trap = d2_trapezoidal(@integrand,-2,2,0,4,100);
built_int = integral2(@integrand,-2,2,0,4);
Simpson = simpson_2d(@integrand,-2,2,0,4,100);
fprintf("Analytical result = %f \n" + ...
    "Built in integral function result = %f \n" + ...
    "Trapezoidal result = %f \n" + ...
    "Simpson Result = %f \n",symbolic,built_int,Trap,Simpson)
```

```
Analytical result = -234.666667
Built in integral function result = -234.666667
Trapezoidal result = -234.675200
Simpson Result = -234.666667
```

We can see that all the results are really close and the built in integral function gave the same result as the analytical solution. The trapezoidal numerical function has a percentage error less than 0% when compare to the analytical and built in integral results. Similarly, the Simpson method also has less than 0% percentage error when compared the other two methods. However, if we want to be really close to the analytical function (and hence the integral function), is better to use the Simpson method.

```
100*abs(Trap - built_int)/built_int ;
100*abs(Simpson- built_int)/built_int;
```

```
function [z] = integrand(x,y)
z = x.^2-3.*y.^2+x.*y.^3;
end

function [I] = d2_trapezoidal(f,ax,bx,ay,by,n)
I = 0;
h=(by-ay)/n;% for the y limits
for i = 0:n
    y = ay + i*h;
    if (i==0 || i==n)
        I = I + segment_trapezoidal(f,y,ax,bx,n);
    else
        I = I + 2*segment_trapezoidal(f,y,ax,bx,n);
    end
end

I = I*h/2;
end

function [I] = segment_trapezoidal(f,y,a,b,n)
I=0; % initial value of integral
h=(b-a)/n;
for i=0:n
    x= a + i*h; % goin from a to b
```

```

if (i==0 || i==n)
    I = I + f(x,y);
else % if is not 0 or n
    I = I + 2*f(x,y);
end
end
I = I*h/2;
end
% simpson:
function [I] = simpson_2d(f,ax,bx,ay,by,n) % "calculate" inner int
    if (mod(n,2)==0)
        I=0;
        h = (by-ay)/n;
        for i =0:n
            y = ay + i*h;
            if (i==0 || i==n)
                I = I + simpson_one_third(f,y,ax,bx,n);
            elseif (mod(i,2)==1)
                I = I + 4*simpson_one_third(f,y,ax,bx,n);
            else
                I = I + 2*simpson_one_third(f,y,ax,bx,n);
            end
        end
        I = I*h/3;
    else
        disp("n has to be even")
    end
end

function [I] = simpson_one_third(f,y,a,b,n) % this calculates outer integral
if (mod(n,2)==0)
    I=0;
    h = (b-a)/n;
    for i =0:n
        x = a + i*h;
        if (i==0 || i==n)
            I = I + f(x,y);
        elseif (mod(i,2)==1)
            I = I + 4*f(x,y);
        else
            I = I + 2*f(x,y);
        end
    end
    I = I*h/3;
else
    disp("n has to be even")
end
end

```

Exercise 7:

```
syms f(x,y,z);
zlow=-1;zup=3;ylow=0;yup=6;xlow=-4;xup=4;
f(x,y,z)= z*(x^3-2*x*y-z);
analytical = double(int(int(int(f,z,zlow,zup),y,ylow,yup),x,xlow,xup));
built_in = integral3(@integrand,xlow,xup,ylow,yup,zlow,zup);
Trapezoidal = d3_trapezoidal(@integrand,xlow,xup,ylow,yup,zlow,zup,20);
Simpson = simpson_3d(@integrand,xlow,xup,ylow,yup,zlow,zup,20);
fprintf("Analytical = %f \n" + ...
        "Buil in integral3 = %f \n" + ...
        "Trapezoidal = %f \n" + ...
        "Simpson = %f \n",analytical,built_in,Trapezoidal,Simpson)
```

```
Analytical = -448.000000
Buil in integral3 = -448.000000
Trapezoidal = -449.280000
Simpson = -448.000000
```

```
abs((analytical-Trapezoidal)/analytical)*100
```

```
ans = 0.2857
```

Comparison: We can see that the Analytical result, the built in integral3 function result, and the Simpson numerical result are the same. The Trapezoidal numerical result is the only result that is different with a percentage error of 0.29%.

```
function [m] = integrand(x,y,z)
m= z.*(x.^3-2*x.*y-z);
end

function [I] = d3_trapezoidal(f,ax,bx,ay,by,az,bz,n)
I = 0;
h=(bz-az)/n;% for the y limits
for i = 0:n
    z = az + i*h;
    if (i==0 || i==n)
        I = I + d2_trapezoidal(f,z,ax,bx,ay,by,n);
    else
        I = I + 2*d2_trapezoidal(f,z,ax,bx,ay,by,n);
    end
end
I = I*h/2;
end

function [I] = d2_trapezoidal(f,z,ax,bx,ay,by,n)
I = 0;
h=(by-ay)/n;% for the y limits
for i = 0:n
    y = ay + i*h;
    if (i==0 || i==n)
        I = I + segment_trapezoidal(f,z,y,ax,bx,n);
    else

```

```

        I = I + 2*segment_trapezoidal(f,z,y,ax,bx,n);
    end
end
I = I*h/2;
end

function [I] = segment_trapezoidal(f,z,y,a,b,n)
I=0; % initial value of integral
h=(b-a)/n;
for i=0:n
    x= a + i*h; % goin from a to b
    if (i==0 || i==n)
        I = I + f(x,y,z);
    else % if is not 0 or n
        I = I + 2*f(x,y,z);
    end
end
I = I*h/2;
end

% Simpson:
function [I] = simpson_3d(f,ax,bx,ay,by,az,bz,n) % "calculate" inner int
if (mod(n,2)==0)
    I=0;
    h = (bz-az)/n;
    for i =0:n
        z = az + i*h;
        if (i==0 || i==n)
            I = I + simpson_one_third2d(f,z,ax,bx,ay,by,n);
        elseif (mod(i,2)==1)
            I = I + 4*simpson_one_third2d(f,z,ax,bx,ay,by,n);
        else
            I = I + 2*simpson_one_third2d(f,z,ax,bx,ay,by,n);
        end
    end
    I = I*h/3;
else
    disp("n has to be even")
end
end

function [I] = simpson_one_third2d(f,z,ax,bx,ay,by,n) % "calculate" inner int
if (mod(n,2)==0)
    I=0;
    h = (by-ay)/n;
    for i =0:n
        y = ay + i*h;
        if (i==0 || i==n)
            I = I + simpson_one_third(f,z,y,ax,bx,n);
        elseif (mod(i,2)==1)
            I = I + 4*simpson_one_third(f,z,y,ax,bx,n);
        else
            I = I + 2*simpson_one_third(f,z,y,ax,bx,n);
        end
    end
end

```

```

    end
    I = I*h/3;
else
    disp("n has to be even")
end
end

function [I] = simpson_one_third(f,z,y,ax,bx,n) % this calculates outer integral
if (mod(n,2)==0)
    I=0;
    h = (bx-ax)/n;
    for i =0:n
        x = ax + i*h;
        if (i==0 || i==n)
            I = I + f(x,y,z);
        elseif (mod(i,2)==1)
            I = I + 4*f(x,y,z);
        else
            I = I + 2*f(x,y,z);
        end
    end
    I = I*h/3;
else
    disp("n has to be even")
end
end

```

Exercise 8:

```
lowerlim = [-4 0 -1];
upperlim = [4 6 3];
a=monte_carlo_gen(@integrand,lowerlim,upperlim,3,1000);
b=monte_carlo_gen(@integrand,lowerlim,upperlim,3,100000);
result = -448;
pea=100*abs((result-a)/result);
peb=100*abs((result-b)/result);
fprintf("n = 1000 Monte Carlo result = %f with percent error %f \n" + ...
        "n=100000 Monte Carlo result = %f with percent error %f ",a,pea,b,peb)
```

```
n = 1000 Monte Carlo result = -417.469088 with percent error 6.814936
n=100000 Monte Carlo result = -461.561339 with percent error 3.027085
```

We can observe that the percentage error of $n=100000$ is smaller than the percentage error of $n=1000$. Therefore our results are more accurate when n is big.

```
function [v] = integrand(x)
v = x(3,:).*(x(1,:).^3-2*x(1,:).*x(2,)-x(3,:));
end

function [I] = monte_carlo_gen(f,lowerlimits,upperlimits,IntegralDimension,n)
x=zeros(IntegralDimension,n); % initialize sampling matrix
mu=prod(upperlimits-lowerlimits); % calculate total volume
y=rand(IntegralDimension,n); % random numbers within [0,1]
% convert to within [a,b]
for i=1:IntegralDimension
    x(i,:)=lowerlimits(i)+(upperlimits(i)-lowerlimits(i))*y(i,:);
end
I=mu*sum(feval(f,x))/n; % evaluate at all points at once
end
```